

Modal and Finite-Difference Methods

Differentiable Physical Modelling and Machine Learning for Sound Synthesis

Rodrigo Diaz

2026-05-14

Week 2 – Modal and Finite-Difference Methods

1

Overview

- Recap and a gentler look at the wave equation
- Separation of variables and basis expansion
- Two ways to discretise the spatial operator
- Modal view: eigenvalue problem and mode shapes
- From modes to state-space ODEs
- **Short coding activity (laptops open)**
- 2D preview: membranes and plates
- FDTD: the same operator on a grid
- Stability and the CFL condition
- Modal vs FDTD: when to use which

Recap and the Wave Equation

Where we left off

Week 1 closed with the run-time recursion:

$$\begin{aligned}\mathbf{h}_{n+1} &= \mathbf{A} \mathbf{h}_n + \mathbf{b} f_n \\ \mathbf{y}_n &= \mathbf{C} \mathbf{h}_n\end{aligned}$$

But we never actually built $\mathbf{A}$.

This week: two recipes to construct $\mathbf{A}$ from the PDE. They differ in the basis we choose to represent the spatial field.

The 1D Wave Equation

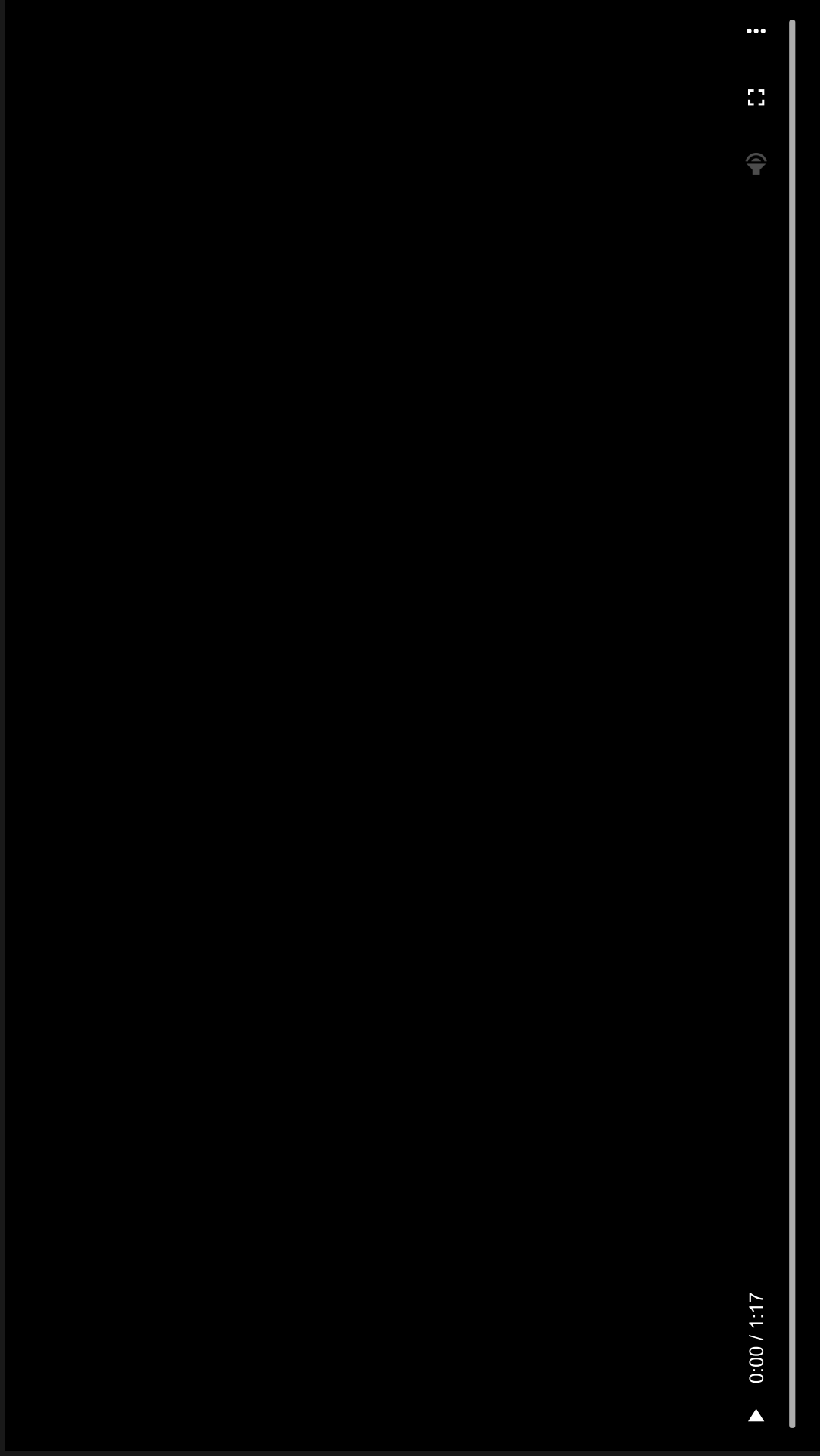
The ideal vibrating string (week 1):

$$\partial_t^2 w = \hat{T} \partial_x^2 w, \quad x \in [0, 1]$$

State is the field $w(x, t)$ – a function of both space and time. Time evolution and spatial shape are **tangled together** in this PDE.

Plan: separate space from time, expand each in a basis of simple functions, then re-assemble.

Separation of Variables



Basis Expansion

The superposition is already a **basis expansion in space**. Rename for the rest of the lecture:

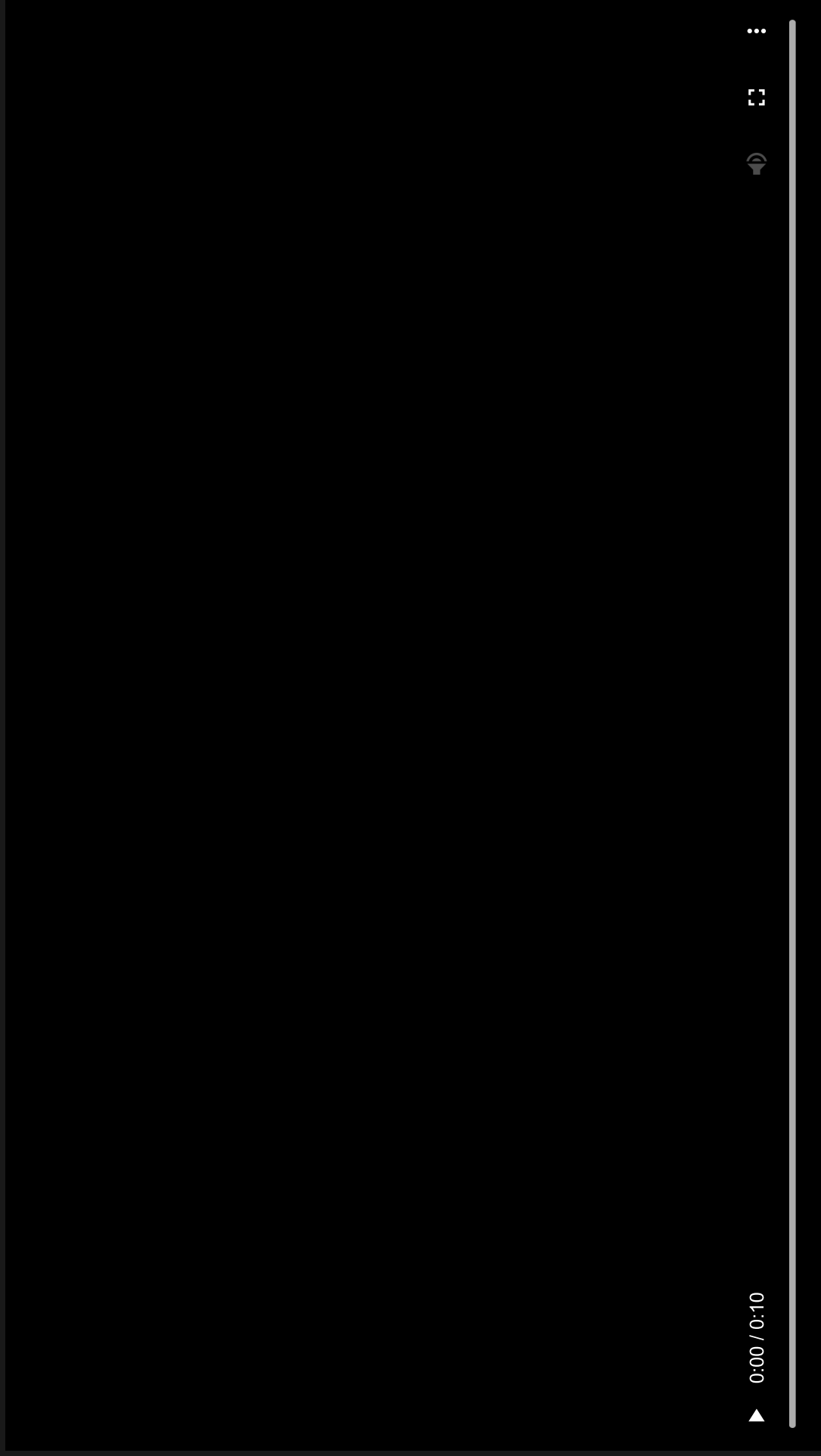
$$F_{\mu}(x) \longrightarrow \phi_{\mu}(x), \quad G_{\mu}(t) \longrightarrow q_{\mu}(t)$$

$$w(x, t) = \sum_{\mu} \phi_{\mu}(x) q_{\mu}(t)$$

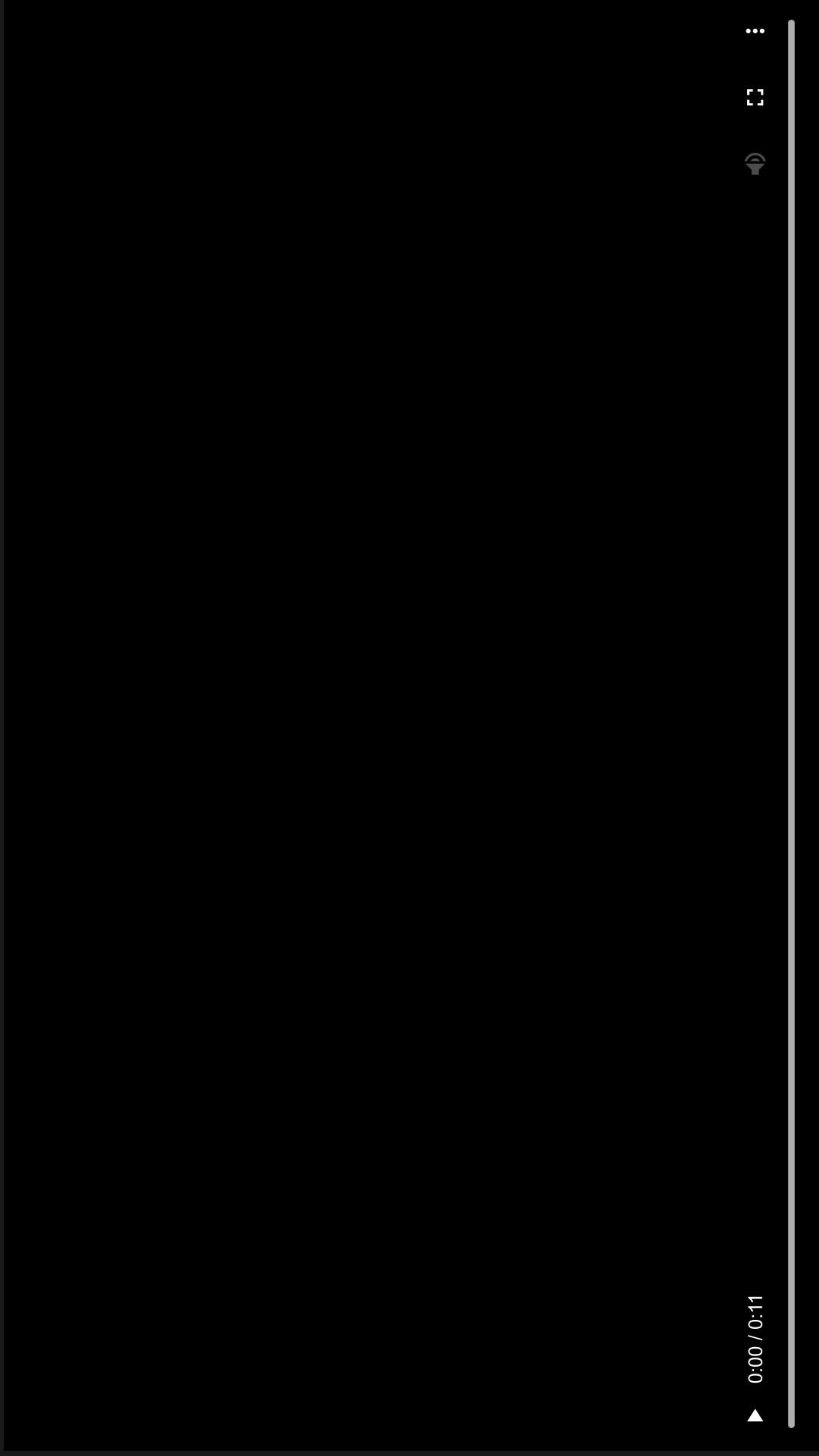
Claim: functions in the relevant function space can be represented exactly with an **infinite** complete basis.

Here: finite-energy string shapes on $[0, L]$ that satisfy the boundary conditions. Smooth functions can be **approximated** with a **finite** set. The smoother the function, the fewer terms you need.

Separation + Basis Expansion



Local vs Global Basis Functions



A Teaser: Learn the Basis

Modal and FDTD both pick a fixed spatial basis. A shallow fully-connected network with one hidden layer is itself a basis expansion:

$$\phi_n(x) = \sum_{j=1}^m A_{nj} \sigma(w_j x + b_j)$$

Activation σ

Basis shape

Character

ReLU

piecewise-linear
segments (3+
neurons build a hat)

local-capable

sin

sinusoid at learned

global-like

frequency w_j

Week 2 – Modal and Finite-Difference Methods

A Teaser: From Network to Modal ODEs

[week02/notebooks/modal_identification.py](#) fits the modal expansion directly to grid data $U \in \mathbb{R}^{N_x \times T}$:

$$u(x, t) = \sum_{n=1}^N \phi_n(x; \theta) \left[e^{-\gamma_n t} (a_n \cos \omega_n t + b_n \sin \omega_n t) \right]$$

Jointly identified:

Two Ways to Discretise

Two Strategies for $\mathcal{L}$

The spatial operator $\mathcal{L} = \hat{T}^2 \partial_x^2$ (later: $\hat{T}^2 \partial_x^2 - \hat{D}^2 \partial_x^4, \Delta, \Delta^2, \dots$) is what we need to represent on a computer.

Project the PDE (Modal)

Expand $w(x, t) = \sum_{\mu} \phi_{\mu}(x) q_{\mu}(t)$ and project the PDE onto $\{\phi_{\mu}\}$.

Pick ϕ_{μ} = eigenfunctions of the spatial derivative operator $\Rightarrow$ operator becomes scalar multiplication by $-\lambda_{\mu}^2$.

State: mode amplitudes $q_{\mu}(t)$.

Today: modal first, FDTD second.

Discretise the operator (FDTD)

Sample the field at grid points x_i and replace ∂_x^2 with a finite-difference stencil.

$\mathcal{L}$ becomes a sparse banded matrix acting on the vector $\mathbf{w}(t) = [w(x_i, t)]$.

State: grid values $w(x_i, t)$.

And Time? Finite Differences Either Way

Both routes leave us with second-order ODEs in time:

- Modal: $\ddot{q}_\mu + 2\gamma_\mu \dot{q}_\mu + \omega_\mu^2 q_\mu = b_\mu f(t)$ (one scalar ODE per mode)
- FDTD: $\ddot{\mathbf{w}}(t) = \hat{T}^2 \mathbf{D}_{xx} \mathbf{w}(t) + \mathbf{b} f(t)$ (coupled grid ODE)

We discretise time the same way we discretised space (in the FDTD case): replace ∂_t^2 with a centred second difference.

Störmer-Verlet / leapfrog for $\ddot{q} = F(q)$:

The Spatial Operator: Modal View

Eigenvalue Problem

We want a basis that makes the spatial derivative operator ∂_x^2 act diagonally. Pick ϕ_μ to satisfy

$$\partial_x^2 \phi_\mu(x) = -\lambda_\mu^2 \phi_\mu(x)$$

[operator] acting on an [eigenfunction] returns the same eigenfunction scaled by an [eigenvalue] (squared, so that λ_μ carries units of a wave number).

Boundary conditions are part of the operator – they pick which ϕ_μ are admissible. Dirichlet, Neumann, mixed all give different spectra.

Ideal Dirichlet string ($\phi_\mu(0) = \phi_\mu(L) = 0$):

$$\phi_\mu(x) = \sin\left(\frac{\mu\pi x}{L}\right), \quad \lambda_\mu = \frac{\mu\pi}{L}, \quad \mu = 1, 2, 3, \dots$$

Mode orthogonality and normalisation

The ϕ_μ are orthogonal under the standard inner product on $[0, L]$:

$$\langle \phi_\mu, \phi_\nu \rangle = \int_0^L \phi_\mu(x) \phi_\nu(x) dx = \|\phi_\mu\|^2 \delta_{\mu\nu}$$

where the squared norm is $\|\phi_\mu\|^2 = L/2$ (for the 1D string).

To **normalise** the modes (so they are **orthonormal**), we divide by $\sqrt{L/2}$. This will come handy when we transform between the physical domain and the modal domain.

Orthogonality is what allows us to extract coefficients independently.

Cross-terms vanish ($\delta_{\mu\nu} = 0$ for $\mu \neq \nu$), so projecting onto ϕ_μ isolates its coefficient.

Generalise the Operator: Add Stiffness

The stiff string from week 1:

$$\partial_t^2 w = \hat{T} \partial_x^2 w - \hat{D} \partial_x^4 w$$

The spatial operator gains a fourth derivative.

Same recipe, same boundary conditions. Sines still diagonalise both pieces:

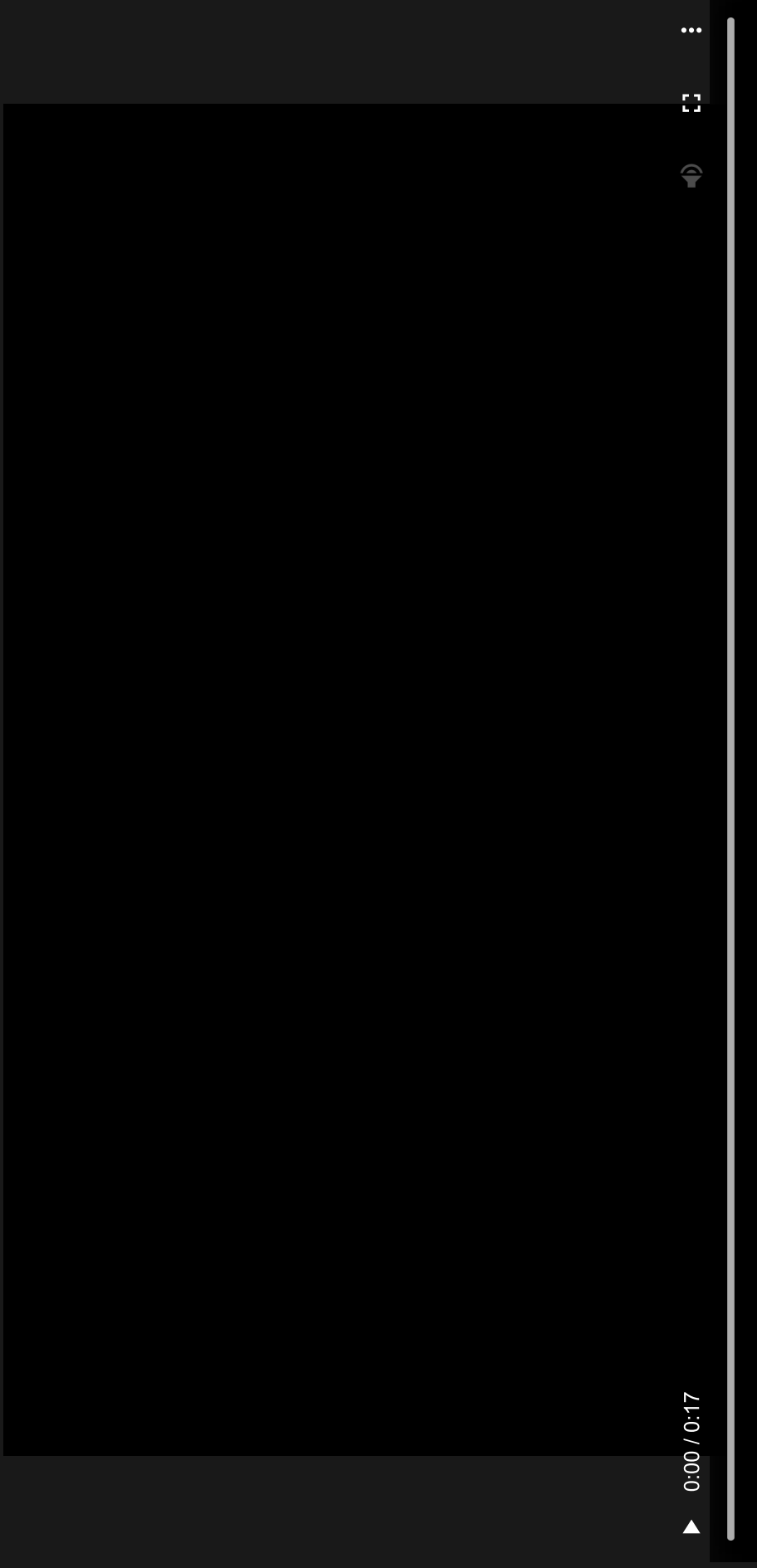
$$\partial_x^2 \phi_\mu = -\lambda_\mu^2 \phi_\mu, \quad \partial_x^4 \phi_\mu = +\lambda_\mu^4 \phi_\mu$$

For simply-supported ends the eigenfunctions are still sines, but the spectrum shifts:

$$\omega_\mu^2 = \hat{T} \lambda_\mu^2 + \hat{D} \lambda_\mu^4, \quad \lambda_\mu = \frac{\mu\pi}{L}$$

The high modes are pushed up disproportionately $\rightarrow$ inharmonicity.

Stiffness → Inharmonicity



This is the inharmonicity we heard in the stage-2 plot of `progressive_string.py` last week.

Modal ODEs and State-space Form

Projection onto the Modal Basis

Inverse transform – sum over the discrete mode index μ :

$$w(x, t) = \sum_{\mu} \phi_{\mu}(x) q_{\mu}(t)$$

Forward transform – integrate the field against each mode:

[Math Processing Error]

Two readings of the same operation:

- Projection of $w(\cdot, t)$ onto ϕ_{μ} , orthogonality of $\{\phi_{\mu}\}$ makes the picture exact.
- Integral transform with kernel $\phi_{\mu}(x)$. Here $\phi_{\mu}(x) = \sin(\mu\pi x/L)$, so this is a sine transform.

Projection onto the Modal Basis

Insert $w(x, t) = \sum_{\mu} \phi_{\mu}(x) q_{\mu}(t)$ into the forced damped wave equation:

$$\sum_{\mu} \phi_{\mu}(x) \ddot{q}_{\mu}(t) = \hat{T}^2 \sum_{\mu} \partial_x^2 \phi_{\mu}(x) q_{\mu}(t) + \dots$$

Use $\partial_x^2 \phi_{\mu} = -\lambda_{\mu}^2 \phi_{\mu}$, take the inner product with ϕ_{ν} , and orthogonality kills the cross-terms:

$$\ddot{q}_{\mu} + 2\gamma_{\mu} \dot{q}_{\mu} + \omega_{\mu}^2 q_{\mu} = b_{\mu} f(t)$$

Quantity	Role
$\omega_{\mu}^2 = \hat{T}^2 \lambda_{\mu}^2 (+\hat{D}^2 \lambda_{\mu}^4)$	Modal frequency squared
$2\gamma_{\mu}$	Modal damping coefficient (next slide)
$b_{\mu} = \phi_{\mu}(x_{\text{in}}) / \ \phi_{\mu}\ ^2$	Input projection

Week 2 – Modal and Finite-Difference Methods

Damping per Mode

Each loss term from week 1 projects cleanly onto the modal basis.

PDE term	Modal contribution to $2\gamma_\mu$
$-\hat{d}_1 \partial_t w$	$2\gamma_\mu \hat{d}_1$
$+\hat{d}_3 \partial_t \partial_x^2 w$	$2\gamma_\mu \hat{d}_3 \lambda_\mu^2$

So $2\gamma_\mu = \hat{d}_1 + \hat{d}_3 \lambda_\mu^2$.

Frequency-dependent damping ($\hat{d}_3$) becomes a per-mode damping coefficient – for free, just from the projection. High modes decay faster, exactly what real strings do.

Output Map and State-space Form

The output at readout position x_{out} is the field sampled at that point:

$$y(t) = w(x_{\text{out}}, t) = \sum_{\mu} \phi_{\mu}(x_{\text{out}}) q_{\mu}(t)$$

Stack the modal amplitudes and their derivatives into a state vector:

[Math Processing Error]

[Math Processing Error]

Here Ω and Γ are real diagonal matrices; complex modal poles appear as eigenvalues of $\mathbf{A}$, not as entries of Ω .

For a point force at x_{in} and one readout at x_{out} :

[Math Processing Error]

$\mathbf{A}$ is block-diagonal in mode pairs. The N modal ODEs decouple completely.

Tie-back: progressive_string.py

Week 1's notebook was already this recipe:

```

1 lambda_mu_squared = string_eigenvalues(n_modes, base_params.length)
2 lambda_mu = np.sqrt(lambda_mu_squared)
3 K = string_eigenfunctions(lambda_mu, grid)

1 gamma2_mu = damping_term(params, lambda_mu_squared) # 2 * gamma_mu
2 omega_mu_squared = stiffness_term(params, lambda_mu_squared) # omega_mu^2

1 _, modal_sol, _ = solve_tf(
2     gamma2_mu, omega_mu_squared,
3     dt=dt, n_steps=n_steps,
4     nl_fn=nl_fn, u0=u0, v0=jnp.zeros_like(u0),
5 )
6 modal_sol = modal_sol.T # (n_modes, n_steps)
7 return readout_weights @ modal_sol

```

Same equations, same state-space, just JAX'd.

Mid-Session Activity

Eigenvalues

- Build a tridiagonal second-difference matrix $\mathbf{D}_{xx}$ with stencil $[1, -2, 1]/\Delta x^2$. Compute its eigenvalues and plot them against mode index. What shape do you see? How does it compare with the closed-form $-\lambda_\mu^2$?

2D Preview: Membranes and Plates

Same Recipe, New Domain

The recipe is identical: solve the eigenvalue problem for the spatial operator on the new domain.

Square Dirichlet membrane ($\partial_t^2 w = \hat{T}^2 \Delta w$ on $[0, L_x] \times [0, L_y]$):

[Math Processing Error]

Separable in 2D $\rightarrow$ eigenfunctions are products of 1D eigenfunctions.

The frequencies are

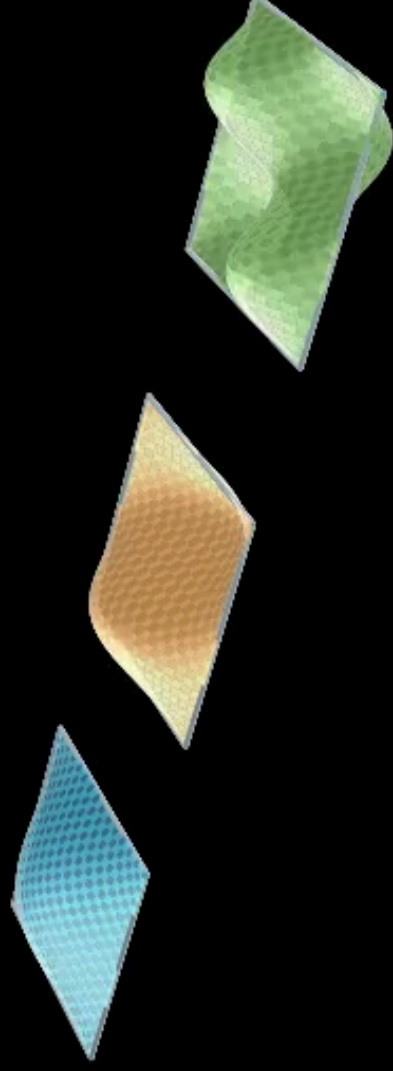
$$\omega_{m,n} = \hat{T} \pi \sqrt{\frac{m^2}{L_x^2} + \frac{n^2}{L_y^2}}$$

so they are **not** integer-spaced in general: they depend on two integer indices through a square root, rather than one integer harmonic number.

Membrane Modes

$$\phi_{m,n}(x, y) = \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right)$$

Square Dirichlet membrane modes



$$(m, n) = (1, 1) \quad \omega/\omega_{1,1} = 1.00 \quad (m, n) = (2, 1) \quad \omega/\omega_{1,1} = 1.58 \quad (m, n) = (2, 2) \quad \omega/\omega_{1,1} = 2.00$$

▶ 0:00 / 0:21



Circular membranes – not separable in (x, y) , but separable in (r, θ) . Eigenfunctions are Bessel functions $J_n(\lambda_{n,k}r) \cos(n\theta)$, with sine and cosine angular pairs for $n > 0$. Familiar

territory: timpani.

Plates

A plate is to a membrane what a stiff string is to an ideal string: bending stiffness dominates.

The spatial operator becomes **biharmonic**:

[Math Processing Error]

For a simply-supported rectangular plate:

[Math Processing Error]

The squared bracket gives the **cymbal-like inharmonicity**: very dense, irregular spectra.
Real-world: gongs, cymbals, drums with a stiff head.

Same recipe – different operator, different spectrum, same code structure.

FDTD: The Grid View

Back to the Spatial Operator

We picked the modal basis to make $\mathcal{L}$ diagonal.

What if we pick a local grid basis instead?

Sample the field at N equispaced points $x_i = (i - 1) \Delta x, i = 1, \dots, N$, with $\Delta x = L/(N - 1)$:

[Math Processing Error]

The degrees of freedom are no longer modal amplitudes $q_\mu(t)$. They are nodal values $w_i(t) = w(x_i, t)$.

If we want a basis-language analogy, this corresponds to local interpolation functions $\Lambda_i(x)$, each centred at grid point x_i .

Discrete Spatial Operator

Approximate $\partial_x^2 w$ by the centred second difference:

$$\partial_x^2 w(x_i, t) \approx \frac{w_{i-1} - 2w_i + w_{i+1}}{\Delta x^2}$$

In matrix form, $\partial_x^2 \mapsto \mathbf{D}_{xx}$ – stencil $\frac{1}{\Delta x^2} [1, -2, 1]$ on the interior rows:

[Math Processing Error]

The spatial operator is now $\mathbf{D}_{xx}$, a sparse banded matrix, not diagonal.

Discrete Spatial Operator

The interior stencil is fixed:

[Math Processing Error]

The boundary rows of $\mathbf{D}_{xx}$ depend on the boundary condition:

Boundary condition

Dirichlet

First and last rows are zero; enforce $w_1^n = w_N^n = 0$ for all n .

Neumann

Mirror ghost points: $w_0 = w_2$,
 $w_{N+1} = w_{N-1}$.

Periodic

Wrap the stencil so $w_0 = w_N$ and
 $w_{N+1} = w_1$.

So $\mathbf{D}_{xx}$ is the interior stencil **plus a boundary closure**, not just the three numbers $[1, -2, 1]$.

Stencil Sliding Along the Grid



Discrete Time: Centred Differences

Approximate $\partial_t^2 w$ the same way – the δ_{tt} operator from week 1’s homework:

$$\partial_t^2 w(x_i, t_n) \approx \frac{w_i^{n-1} - 2w_i^n + w_i^{n+1}}{\Delta t^2}$$

Substitute into the wave equation and solve for $\mathbf{w}^{n+1}$:

[Math Processing Error]

A two-step recursion. The Courant number $\nu := \hat{T} \Delta t / \Delta x$ controls everything about the scheme.

Output Map and State-space Form

Stack $\mathbf{w}^n$ and $\mathbf{w}^{n-1}$ into a state vector:

[Math Processing Error]

[Math Processing Error]

For a point force injected at x_{in} , let $\mathbf{g}_{\text{in}}$ be the vector of local interpolation weights,

$$(\mathbf{g}_{\text{in}})_i = \Lambda_i(x_{\text{in}}):$$

[Math Processing Error]

For one readout at x_{out} , use the same interpolation idea: $(\mathbf{g}_{\text{out}})_i = \Lambda_i(x_{\text{out}})$.

[Math Processing Error]

Same shape as the modal recursion. $\mathbf{A}$ is sparse-banded instead of block-diagonal. The state-space machinery from week 1 carries over unchanged.

Stability: the CFL Condition

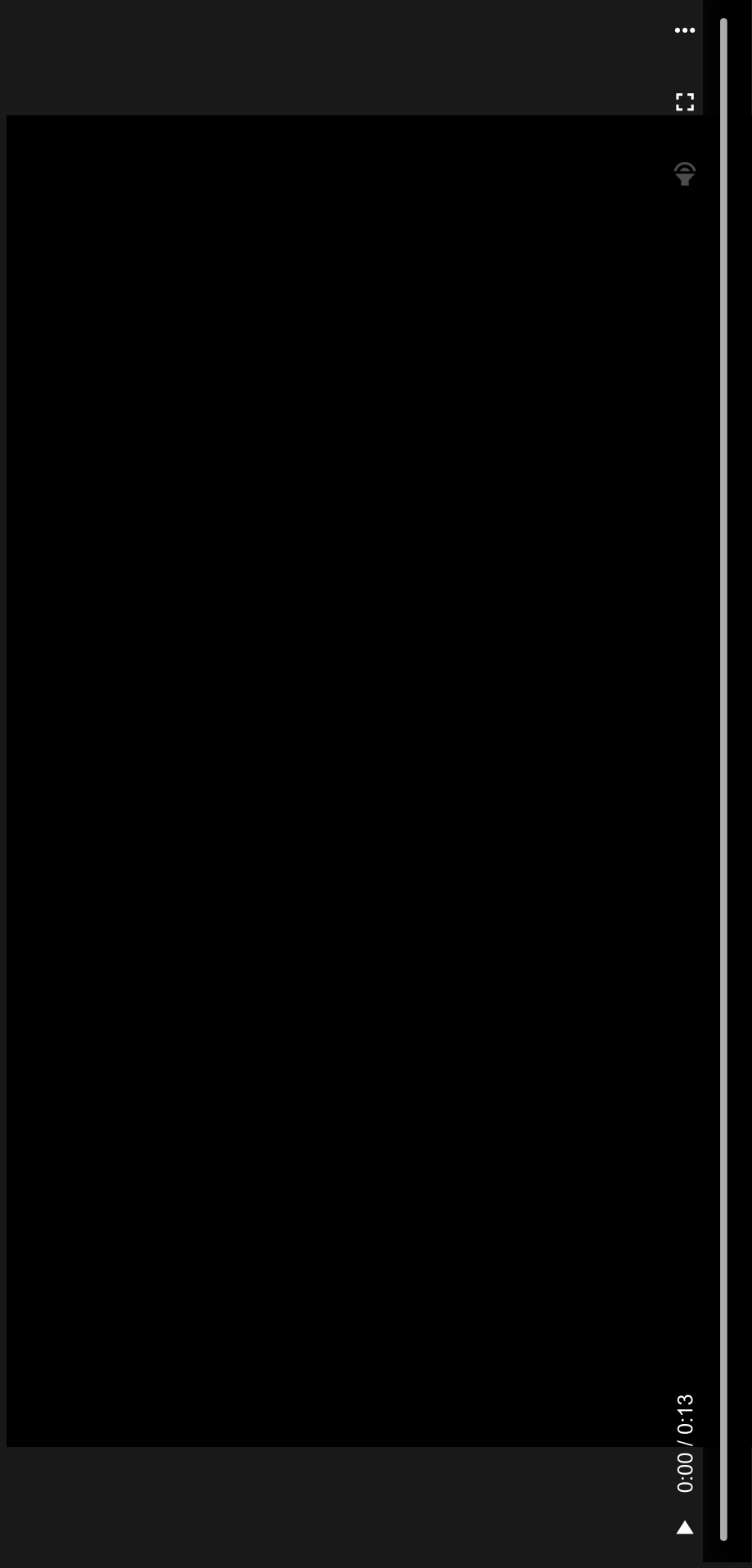
The eigenvalues of D_{xx} determine when the explicit scheme blows up.

Plugging an eigenvector $\mathbf{v}_k$ into the recursion and demanding $|z_k| \leq 1$ for the per-mode amplification factor yields the **Courant–Friedrichs–Lewy condition**:

$$\nu = \hat{T} \frac{\Delta t}{\Delta x} \leq 1$$

- $\nu < 1$: stable, but extra numerical dispersion.
- $\nu = 1$: the “magic” choice – the discrete scheme is exact for the ideal wave equation.
- $\nu > 1$: catastrophic blow-up within milliseconds.

CFL Blow-up



Dispersion in the High Modes

Let λ be the continuous wavenumber, so $\lambda = \lambda_\mu$ for modal mode μ .

The modal stiff-string dispersion relation is the physical one:

$$\omega_{\text{modal}}^2(\lambda) = \hat{T}^2 \lambda^2 + \hat{D}^2 \lambda^4$$

For the ideal-string FDTD scheme ($\hat{D} = 0$), centred differences give the numerical relation

[Math Processing Error]

Low modes ($\lambda \Delta x \ll 1$): $\sin(\lambda \Delta x / 2) \approx \lambda \Delta x / 2$, so $\omega_{\text{discrete}} \approx \hat{T} \lambda$, matching the ideal modal relation.

High modes: modal stiffness adds the physical upward bend $+\hat{D}^2 \lambda^4$, while FDTD adds numerical dispersion because the grid replaces λ^2 by

Week 2 – Modal and Finite-Difference Methods

Modal vs FDTD

State-space Side by Side

Modal: continuous time

[Math Processing Error]

A matrix

[Math Processing Error]

Diagonal mode pairs: modes are decoupled.

Input map

[Math Processing Error]

Input is projected onto eigenfunctions.

Output map

[Math Processing Error]

Output samples the modal expansion.

Same state-space ingredients, but not the same time domain yet: modal is still an ODE, FDTD is already a recurrence.

FDTD: discrete time

[Math Processing Error]

A matrix

[Math Processing Error]

Sparse grid coupling through D_{xx} .

Input map

[Math Processing Error]

Input is distributed to nearby grid points.

Output map

[Math Processing Error]

Output interpolates from local grid values.

State-space Side by Side

Time-discrete modal

Modal: discrete time

[Math Processing Error]

A matrix

Using centred differences for both $\ddot{\mathbf{q}}$ and $\dot{\mathbf{q}}$, let

$$\mathbf{R} = (\mathbf{I} + \Delta t \mathbf{\Gamma})^{-1};$$

[Math Processing Error]

Still diagonal mode pairs: the inverse is elementwise, and modes remain decoupled.

Input map

[Math Processing Error]

Input is projected onto eigenfunctions.

Output map

[Math Processing Error]

Output samples the modal expansion.

Week 2 – Modal and Finite-Difference Methods

FDTD: discrete time

[Math Processing Error]

A matrix

[Math Processing Error]

Sparse grid coupling through $\mathbf{D}_{xx}$.

Input map

[Math Processing Error]

Input is distributed to nearby grid points.

Output map

[Math Processing Error]

Output interpolates from local grid values.

When to Use Which

Week 2 – Modal and Finite-Difference Methods

Wrapping Up

Next Week

Session 3: Nonlinear Systems and Coupled Models

- Main focus: modal treatment of nonlinearities and mode coupling.
- Collisions and excitation functions: brief tour of force laws and input maps.
- Coupled subsystems: string + body
- Differentiability: why it becomes interesting once nonlinearities are in.

Homework

Reading

- Bilbao, *Numerical Sound Synthesis*, Sec. 8.1, “The Kirchhoff–Carrier string model”.

Optional

- As we look further next week into optimising parameters for physical and neural networks, we face many obstacles. Try to optimise the argument of the `sin` function. This can be done in 10 or so lines with JAX. What do you observe? Do you always converge to the right true value? Try with different initialisations.
- Try to make sense on your own of [week02/notebooks/modal_identification.py](#). We will look at JAX later on and how the autodiff works, but for now try to change some parameters and try to see where it fails.